In [ ]:
```
# pip installs
!pip install requests lyricsgenius pandas
```

```
Collecting requests
  Downloading requests-2.32.5-py3-none-any.whl.metadata (4.9 kB)
Collecting lyricsgenius
  Using cached lyricsgenius-3.7.5-py3-none-any.whl.metadata (6.2 kB)
Collecting pandas
  Downloading pandas-2.3.3-cp314-cp314-macosx_11_0_arm64.whl.metadata (91 k
B)
Collecting charset_normalizer<4,>=2 (from requests)
  Downloading charset_normalizer-3.4.4-cp314-cp314-macosx_10_13_universal2.w
hl.metadata (37 kB)
Collecting idna<4,>=2.5 (from requests)
  Downloading idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Collecting urllib3<3,>=1.21.1 (from requests)
  Downloading urllib3-2.6.3-py3-none-any.whl.metadata (6.9 kB)
Collecting certifi>=2017.4.17 (from requests)
  Downloading certifi-2026.1.4-py3-none-any.whl.metadata (2.5 kB)
Collecting beautifulsoup4>=4.12.3 (from lyricsgenius)
  Using cached beautifulsoup4-4.14.3-py3-none-any.whl.metadata (3.8 kB)
Collecting numpy>=1.26.0 (from pandas)
  Downloading numpy-2.4.1-cp314-cp314-macosx_14_0_arm64.whl.metadata (6.6 k
B)
Requirement already satisfied: python-dateutil>=2.8.2 in ./.venv/lib/python
3.14/site-packages (from pandas) (2.9.0.post0)
Collecting pytz>=2020.1 (from pandas)
  Using cached pytz-2025.2-py2.py3-none-any.whl.metadata (22 kB)
Collecting tzdata>=2022.7 (from pandas)
  Downloading tzdata-2025.3-py2.py3-none-any.whl.metadata (1.4 kB)
Collecting soupsieve>=1.6.1 (from beautifulsoup4>=4.12.3->lyricsgenius)
  Using cached soupsieve-2.8.1-py3-none-any.whl.metadata (4.6 kB)
Collecting typing-extensions>=4.0.0 (from beautifulsoup4>=4.12.3->lyricsgeni
us)
  Downloading typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Requirement already satisfied: six>=1.5 in ./.venv/lib/python3.14/site-packa
ges (from python-dateutil>=2.8.2->pandas) (1.17.0)
Downloading requests-2.32.5-py3-none-any.whl (64 kB)
Downloading charset_normalizer-3.4.4-cp314-cp314-macosx_10_13_universal2.whl
(207 kB)
Downloading idna-3.11-py3-none-any.whl (71 kB)
Downloading urllib3-2.6.3-py3-none-any.whl (131 kB)
Using cached lyricsgenius-3.7.5-py3-none-any.whl (48 kB)
Downloading pandas-2.3.3-cp314-cp314-macosx_11_0_arm64.whl (10.8 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 10.8/10.8 MB 12.5 MB/s  0:00:00
eta 0:00:01
Using cached beautifulsoup4-4.14.3-py3-none-any.whl (107 kB)
Downloading certifi-2026.1.4-py3-none-any.whl (152 kB)
Downloading numpy-2.4.1-cp314-cp314-macosx_14_0_arm64.whl (5.2 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 5.2/5.2 MB 11.8 MB/s  0:00:00 et
a 0:00:01
Using cached pytz-2025.2-py2.py3-none-any.whl (509 kB)
Using cached soupsieve-2.8.1-py3-none-any.whl (36 kB)
Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
Downloading tzdata-2025.3-py2.py3-none-any.whl (348 kB)
Installing collected packages: pytz, urllib3, tzdata, typing-extensions, sou
psieve, numpy, idna, charset_normalizer, certifi, requests, pandas, beautifu
lsoup4, lyricsgenius
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 13/13 [lyricsgenius][0m [panda
```

```
s]]
Successfully installed beautifulsoup4-4.14.3 certifi-2026.1.4 charset_normal
izer-3.4.4 idna-3.11 lyricsgenius-3.7.5 numpy-2.4.1 pandas-2.3.3 pytz-2025.2
requests-2.32.5 soupsieve-2.8.1 typing-extensions-4.15.0 tzdata-2025.3 urlli
b3-2.6.3
```

In [ ]:
```
import requests
import lyricsgenius
import re
import pandas as pd
import time
from datetime import datetime

# key
GENIUS_ACCESS_TOKEN = 'Skzt90n0y5zzpwsx9wnC07RxPL16BLRQmz0xfTE320wSVulgjKKVh
```

In [ ]:
```
genius = lyricsgenius.Genius(GENIUS_ACCESS_TOKEN)
genius.verbose = False
genius.remove_section_headers = True

print("Genius API initialized successfully!")
```

```
Genius API initialized successfully!
iTunes API requires no setup - ready to use!
```

In [ ]:
```
# methods
VALID_GENRES = ["pop", "rock", "hiphop", "rb", "country", "jazz", "edm",
                "classical", "kpop", "latin", "folk", "metal", "reggae", "ot

def clean_text_field(text):
    if not text:
        return ""
    text = text.lower()
    text = re.sub(r"[^a-z0-9\s,.]", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

def clean_lyrics_strict(text):
    if not text:
        return ""
    text = text.lower()
    text = text.replace("\n", " ")
    text = re.sub(r"[^a-z0-9\s,.]", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text

def map_itunes_genre(itunes_genre):
    if not itunes_genre:
        return "other"

    g = itunes_genre.lower()
    if "k-pop" in g or "kpop" in g:
        return "kpop"
    if "hip-hop" in g or "rap" in g or "hip hop" in g:
        return "hiphop"
    if "r&b" in g or "soul" in g or "rnb" in g:
```

```
        return "rb"
    if "metal" in g:
        return "metal"
    if "rock" in g or "alternative" in g:
        return "rock"
    if "country" in g:
        return "country"
    if "jazz" in g:
        return "jazz"
    if "classical" in g:
        return "classical"
    if "folk" in g or "singer/songwriter" in g:
        return "folk"
    if "reggae" in g:
        return "reggae"
    if "dance" in g or "electronic" in g or "house" in g or "edm" in g:
        return "edm"
    if "latin" in g or "latino" in g or "urbano" in g or "reggaeton" in g:
        return "latin"
    if "pop" in g:
        return "pop"

    return "other"

print("Functions defined successfully!")
```

```
Functions defined successfully!
```

In [ ]:

```
# data fetching
def get_itunes_candidates():
    candidates = []

    search_terms = [
        "Taylor Swift", "Drake", "The Weeknd", "Bad Bunny", "Morgan Wallen",
        "SZA", "Doja Cat", "Post Malone", "Billie Eilish", "Dua Lipa",
        "Kendrick Lamar", "Travis Scott", "Olivia Rodrigo", "Sabrina Carpent
        "Ariana Grande", "Bruno Mars", "Ed Sheeran", "Harry Styles", "Beyonc
        "Rihanna", "Justin Bieber", "Miley Cyrus", "Lady Gaga", "Kanye West"
        "Future", "21 Savage", "Lil Baby", "Gunna", "Tyler the Creator",
        "Lana Del Rey", "Hozier", "Chappell Roan", "Tate McRae", "Gracie Abr
        "Jack Harlow", "Metro Boomin", "Nicki Minaj", "Cardi B", "Megan Thee
        "Peso Pluma", "Feid", "Karol G", "Shakira", "Rosalia",
        "BTS", "BLACKPINK", "NewJeans", "Stray Kids", "aespa",
        "Coldplay", "Imagine Dragons", "OneRepublic", "Maroon 5",
        "top hits 2025", "new music 2025", "2025 songs", "billboard 2025",
        "pop 2025", "rap 2025", "country 2025", "latin 2025"
    ]

    print("Fetching metadata from iTunes API...")
    print(f"Searching {len(search_terms)} terms for 2025 songs...\n")

    for term in search_terms:
        try:
            url = "https://itunes.apple.com/search"
            params = {
                "term": term,
```

```
                "media": "music",
                "entity": "song",
                "limit": 200,
                "country": "US"
            }
            response = requests.get(url, params=params, timeout=10)
            data = response.json()

            count_2025 = 0
            for item in data.get('results', []):
                release_date_raw = item.get('releaseDate', '1900-01-01')
                release_date = release_date_raw.split("T")[0]

                if release_date >= "2025-01-01":
                    candidates.append({
                        "title": item.get('trackName', 'Unknown'),
                        "artist": item.get('artistName', 'Unknown'),
                        "date": release_date,
                        "genre": item.get('primaryGenreName', 'other')
                    })
                    count_2025 += 1

            if count_2025 > 0:
                print(f"  '{term}': {count_2025} songs from 2025+")
            time.sleep(0.2)

        except Exception as e:
            print(f"  Error '{term}': {e}")

    unique_candidates = []
    seen = set()
    for c in candidates:
        key = (c['title'].lower(), c['artist'].lower())
        if key not in seen:
            seen.add(key)
            unique_candidates.append(c)

    print(f"\n==> Found {len(unique_candidates)} unique candidates from 2025
    return unique_candidates

candidates = get_itunes_candidates()
```

```
Fetching metadata from iTunes API...
Searching 61 terms for 2025 songs...

  'Taylor Swift': 6 songs from 2025+
  'Drake': 4 songs from 2025+
  'The Weeknd': 20 songs from 2025+
  'Bad Bunny': 8 songs from 2025+
  'Morgan Wallen': 43 songs from 2025+
  'SZA': 19 songs from 2025+
  'Doja Cat': 11 songs from 2025+
  'Post Malone': 3 songs from 2025+
  'Billie Eilish': 6 songs from 2025+
  'Dua Lipa': 2 songs from 2025+
  'Kendrick Lamar': 3 songs from 2025+
  'Travis Scott': 3 songs from 2025+
  'Olivia Rodrigo': 3 songs from 2025+
  'Sabrina Carpenter': 36 songs from 2025+
  'Ariana Grande': 5 songs from 2025+
  'Bruno Mars': 4 songs from 2025+
  'Ed Sheeran': 7 songs from 2025+
  'Harry Styles': 2 songs from 2025+
  'Rihanna': 3 songs from 2025+
  'Justin Bieber': 20 songs from 2025+
  'Miley Cyrus': 17 songs from 2025+
  'Lady Gaga': 14 songs from 2025+
  'Future': 3 songs from 2025+
  '21 Savage': 4 songs from 2025+
  'Gunna': 13 songs from 2025+
  'Tyler the Creator': 19 songs from 2025+
  'Lana Del Rey': 3 songs from 2025+
  'Hozier': 5 songs from 2025+
  'Chappell Roan': 54 songs from 2025+
  'Tate McRae': 60 songs from 2025+
  'Gracie Abrams': 24 songs from 2025+
  'Jack Harlow': 5 songs from 2025+
  'Metro Boomin': 4 songs from 2025+
  'Cardi B': 49 songs from 2025+
  'Megan Thee Stallion': 2 songs from 2025+
  'Peso Pluma': 14 songs from 2025+
  'Feid': 17 songs from 2025+
  'Karol G': 20 songs from 2025+
  'Shakira': 7 songs from 2025+
  'Rosalia': 21 songs from 2025+
  'BTS': 5 songs from 2025+
  'BLACKPINK': 25 songs from 2025+
  'NewJeans': 55 songs from 2025+
  'Stray Kids': 20 songs from 2025+
  'aespa': 41 songs from 2025+
  'Coldplay': 1 songs from 2025+
  'Imagine Dragons': 1 songs from 2025+
  'OneRepublic': 5 songs from 2025+
  'Maroon 5': 2 songs from 2025+
  'top hits 2025': 18 songs from 2025+
  'new music 2025': 19 songs from 2025+
  '2025 songs': 24 songs from 2025+
  'billboard 2025': 49 songs from 2025+
```

```
 'pop 2025': 19 songs from 2025+
 'rap 2025': 14 songs from 2025+
 'country 2025': 33 songs from 2025+
 'latin 2025': 23 songs from 2025+

==> Found 699 unique candidates from 2025+
```

In [ ]:

```
# ranking
dataset = []
rank = 1
skipped_no_lyrics = 0
skipped_short = 0
skipped_error = 0
skipped_translation = 0

REQUEST_DELAY = 3

TRANSLATION_INDICATORS = [
    "traducciones",
    "traductions",
    "traducao",
    "translation",
    "traduccion",
    "ubersetzung",
]

def is_translation_page(url):
    """Check if the Genius URL is a translation page (not original lyrics)."
    if not url:
        return False
    url_lower = url.lower()
    return any(indicator in url_lower for indicator in TRANSLATION_INDICATOR

print("Starting lyrics fetch and cleaning...")
print(f"Using {REQUEST_DELAY}s delay between requests to avoid rate limiting
print("Filtering out translation pages to get original lyrics only.")
print("This will take ~25-30 minutes for 50 songs. Please wait...\n")

for song in candidates:
    if rank > 50:
        break

    try:
        print(f"  [{rank}/50] Searching: {song['title'][:40]}...", end=" ",

        song_genius = genius.search_song(song['title'], song['artist'])

        if not song_genius or not song_genius.lyrics:
            print("NO LYRICS")
            skipped_no_lyrics += 1
            time.sleep(REQUEST_DELAY)
            continue

        if is_translation_page(song_genius.url):
            print(f"TRANSLATION PAGE - skipping")
            skipped_translation += 1
```

```
            time.sleep(REQUEST_DELAY)
            continue

        if len(song_genius.lyrics) < 50:
            print("TOO SHORT")
            skipped_short += 1
            time.sleep(REQUEST_DELAY)
            continue

        clean_text = clean_lyrics_strict(song_genius.lyrics)

        if len(clean_text) < 100:
            print("CLEANED TOO SHORT")
            skipped_short += 1
            time.sleep(REQUEST_DELAY)
            continue

        mapped_genre = map_itunes_genre(song['genre'])
        clean_title = clean_text_field(song['title'])
        clean_artist = clean_text_field(song['artist'])

        spanish_artists = ["bad bunny", "peso pluma", "feid", "karol g", "sh
        artist_lower = song['artist'].lower()
        language = "spanish" if any(sa in artist_lower for sa in spanish_art

        entry = {
            "rank": rank,
            "title": clean_title,
            "artist": clean_artist,
            "release_date": song['date'],
            "genre": mapped_genre,
            "language": language,
            "lyrics_clean": clean_text,
            "source": song_genius.url
        }

        dataset.append(entry)
        print(f"SUCCESS! ({language})")
        rank += 1

        time.sleep(REQUEST_DELAY)

    except Exception as e:
        if "429" in str(e):
            print(f"RATE LIMITED - waiting 60s...")
            time.sleep(60)
        else:
            print(f"ERROR: {str(e)[:50]}")
            skipped_error += 1
            time.sleep(REQUEST_DELAY)

print(f"\n{'='*50}")
print(f"Collection complete!")
print(f"  Songs collected: {len(dataset)}")
print(f"  Skipped (no lyrics): {skipped_no_lyrics}")
print(f"  Skipped (too short): {skipped_short}")
```

```
print(f"  Skipped (translation pages): {skipped_translation}")
print(f"  Skipped (errors): {skipped_error}")
```

```
Starting lyrics fetch and cleaning...
Using 3s delay between requests to avoid rate limiting.
Filtering out translation pages to get original lyrics only.
This will take ~25-30 minutes for 50 songs. Please wait...

  [1/50] Searching: The Fate of Ophelia... SUCCESS! (english)
  [2/50] Searching: Opalite... SUCCESS! (english)
  [3/50] Searching: Elizabeth Taylor... SUCCESS! (english)
  [4/50] Searching: PIMMIE'S DILEMMA... TRANSLATION PAGE - skipping
  [4/50] Searching: GREEDY... SUCCESS! (english)
  [5/50] Searching: MEET YOUR PADRE... TRANSLATION PAGE - skipping
  [5/50] Searching: NOKIA... SUCCESS! (english)
  [6/50] Searching: The Abyss... TRANSLATION PAGE - skipping
  [6/50] Searching: Baptized In Fear... SUCCESS! (english)
  [7/50] Searching: Wake Me Up... TRANSLATION PAGE - skipping
  [7/50] Searching: Given Up On Me... SUCCESS! (english)
  [8/50] Searching: Open Hearts... SUCCESS! (english)
  [9/50] Searching: Take Me Back To LA... SUCCESS! (english)
  [10/50] Searching: Opening Night... SUCCESS! (english)
  [11/50] Searching: Without a Warning... SUCCESS! (english)
  [12/50] Searching: Drive... SUCCESS! (english)
  [13/50] Searching: Give Me Mercy... SUCCESS! (english)
  [14/50] Searching: Big Sleep... TRANSLATION PAGE - skipping
  [14/50] Searching: Hurry Up Tomorrow... SUCCESS! (english)
  [15/50] Searching: Red Terror... SUCCESS! (english)
  [16/50] Searching: Cry For Me... SUCCESS! (english)
  [17/50] Searching: NUEVAYoL... SUCCESS! (spanish)
  [18/50] Searching: WELTiTA... TRANSLATION PAGE - skipping
  [18/50] Searching: CAFé CON RON... TRANSLATION PAGE - skipping
  [18/50] Searching: TURiSTA... SUCCESS! (spanish)
  [19/50] Searching: LO QUE LE PASÓ A HAWAii... SUCCESS! (spanish)
  [20/50] Searching: Bad Bunny... SUCCESS! (english)
  [21/50] Searching: Bad Bunny... NO LYRICS
  [21/50] Searching: Bad Bunny... NO LYRICS
  [21/50] Searching: What I Want... TRANSLATION PAGE - skipping
  [21/50] Searching: I'm The Problem... SUCCESS! (english)
  [22/50] Searching: I Got Better... SUCCESS! (english)
  [23/50] Searching: Just In Case... SUCCESS! (english)
  [24/50] Searching: 20 Cigarettes... SUCCESS! (english)
  [25/50] Searching: Superman... SUCCESS! (english)
  [26/50] Searching: I'm A Little Crazy... SUCCESS! (english)
  [27/50] Searching: TN... SUCCESS! (english)
  [28/50] Searching: I Ain't Comin' Back... NO LYRICS
  [28/50] Searching: Where'd That Girl Go... SUCCESS! (english)
  [29/50] Searching: Dark Til Daylight... SUCCESS! (english)
  [30/50] Searching: Kiss Her In Front Of You... SUCCESS! (english)
  [31/50] Searching: Don't We... SUCCESS! (english)
  [32/50] Searching: Falling Apart... SUCCESS! (english)
  [33/50] Searching: Missing... SUCCESS! (english)
  [34/50] Searching: Leavin's The Least I Could Do... SUCCESS! (english)
  [35/50] Searching: Genesis... SUCCESS! (english)
  [36/50] Searching: Whiskey In Reverse... SUCCESS! (english)
  [37/50] Searching: Come Back As A Redneck... NO LYRICS
  [37/50] Searching: Jack and Jill... SUCCESS! (english)
  [38/50] Searching: Working Man's Song... SUCCESS! (english)
  [39/50] Searching: Drinking Til It Does... SUCCESS! (english)
```

```
  [40/50] Searching: Revelation... SUCCESS! (english)
  [41/50] Searching: Number 3 and Number 7... SUCCESS! (english)
  [42/50] Searching: The Dealer... NO LYRICS
  [42/50] Searching: Nothin' Left... SUCCESS! (english)
  [43/50] Searching: LA Night... SUCCESS! (english)
  [44/50] Searching: Crazy Eyes... SUCCESS! (english)
  [45/50] Searching: Graveyard Whistling... SUCCESS! (english)
  [46/50] Searching: 30 For 30 (with Kendrick Lamar)... NO LYRICS
  [46/50] Searching: BMF... SUCCESS! (english)
  [47/50] Searching: Sza... NO LYRICS
  [47/50] Searching: Drive... SUCCESS! (english)
  [48/50] Searching: Open Arms (just SZA)... SUCCESS! (english)
  [49/50] Searching: Chill Baby... SUCCESS! (english)
  [50/50] Searching: Crybaby... SUCCESS! (english)

==================================================
Collection complete!
  Songs collected: 50
  Skipped (no lyrics): 7
  Skipped (too short): 0
  Skipped (translation pages): 8
  Skipped (errors): 0
```

In [ ]:

```python
# validation and data summary
df = pd.DataFrame(dataset)
print("Dataset Preview:")
print(f"Total songs: {len(df)}")
print(f"\nGenre distribution:")
print(df['genre'].value_counts())
print(f"\nFirst 10 entries:")
df[['rank', 'title', 'artist', 'genre', 'release_date']].head(10)
```

```
Dataset Preview:
Total songs: 50

Genre distribution:
genre
country    25
rb         18
pop         3
latin       3
hiphop      1
Name: count, dtype: int64

First 10 entries:
```

Out[ ]:

	rank	title	artist	genre	release_date
0	1	the fate of ophelia	taylor swift	pop	2025-10-03
1	2	opalite	taylor swift	pop	2025-10-03
2	3	elizabeth taylor	taylor swift	pop	2025-10-03
3	4	greedy	partynextdoor drake	rb	2025-02-14
4	5	nokia	drake	rb	2025-02-14
5	6	baptized in fear	the weeknd	rb	2025-01-31
6	7	given up on me	the weeknd	rb	2025-01-31
7	8	open hearts	the weeknd	rb	2025-01-31
8	9	take me back to la	the weeknd	rb	2025-01-31
9	10	opening night	the weeknd	rb	2025-01-31

In [ ]:

```
# export function
output_file = "single.txt"

with open(output_file, "w", encoding="utf-8") as f:
    for item in dataset:
        line = f"{item['rank']}||{item['title']}||{item['artist']}||{item['r
        f.write(line + "\n")

print(f"Success! File '{output_file}' created with {len(dataset)} entries.")
print(f"\nFile location: {output_file}")
```

```
Success! File 'single.txt' created with 50 entries.

File location: single.txt
```

In [ ]:

```
# file validation
valid_genres = {"pop", "rock", "hiphop", "rb", "country", "jazz", "edm",
                "classical", "kpop", "latin", "folk", "metal", "reggae", "ot

allowed_pattern = re.compile(r'^[a-z0-9\s,.]*$')

errors = []
warnings = []

with open("single.txt", "r", encoding="utf-8") as f:
    lines = f.readlines()

print(f"Total lines: {len(lines)} (expected: 50)")
if len(lines) != 50:
    warnings.append(f"Expected 50 lines, got {len(lines)}")

for i, line in enumerate(lines):
    parts = line.strip().split("||")

    if len(parts) != 8:
        errors.append(f"Line {i+1}: Wrong field count ({len(parts)} instead
```

```
        continue

    rank, title, artist, r_date, genre, lang, lyrics, source = parts

    if not allowed_pattern.match(title):
        errors.append(f"Line {i+1}: Title contains invalid characters: '{tit
    if title != title.lower():
        errors.append(f"Line {i+1}: Title not lowercase: '{title}'")

    if not allowed_pattern.match(artist):
        errors.append(f"Line {i+1}: Artist contains invalid characters: '{ar
    if artist != artist.lower():
        errors.append(f"Line {i+1}: Artist not lowercase: '{artist}'")

    if r_date < "2025-01-01":
        errors.append(f"Line {i+1}: Date {r_date} is before 2025")

    genres = genre.split(",")
    for g in genres:
        if g not in valid_genres:
            errors.append(f"Line {i+1}: Invalid genre '{g}'")

    if not allowed_pattern.match(lyrics):
        bad_chars = [c for c in lyrics if not re.match(r'[a-z0-9\s,.]', c)]
        errors.append(f"Line {i+1}: Lyrics contain illegal characters: {set(

    if len(lyrics.strip()) < 50:
        warnings.append(f"Line {i+1}: Lyrics very short (possibly instrument

if errors:
    print("\n--- ERRORS (must fix) ---")
    for err in errors:
        print(f"  [X] {err}")

if warnings:
    print("\n--- WARNINGS (review) ---")
    for warn in warnings:
        print(f"  [!] {warn}")

if not errors and not warnings:
    print("\nVALIDATION PASSED! File is ready for submission.")
elif not errors:
    print("\nNo critical errors. Review warnings above.")
```

```
Total lines: 50 (expected: 50)

VALIDATION PASSED! File is ready for submission.
```